



Tecnológico de Monterrey

Campus Querétaro

Evidencia Generación y Limpieza de Gramática

Inaki Mancera Llano - A01708827

Materia: Implementacion de Metodos Computacionales

Fecha de entrega
28 de Mayo 2026

1. Contexto del lenguaje.

El lenguaje seleccionado es un subconjunto de JavaScript, específicamente asignaciones con expresiones aritméticas.

No se trabajará JavaScript completo, sino una parte pequeña del lenguaje que permite escribir instrucciones como:

$x = 8 * (a + b);$

Dicho subconjunto cuenta con identificadores (**x, y, resultado, etc.**), números (**1, 2, 3, 4, etc.**), operadores (**+, -, *, /**), paréntesis (**()**), asignación de valor (**=**) y fin de una instrucción (**;**).

1.1 Gramática:

JavaScript es un lenguaje de programación ampliamente usado en desarrollo web. Una parte básica de su sintaxis consiste en declarar o modificar valores mediante asignaciones y expresiones.

Estas asignaciones tienen una estructura sintáctica como **id = expresion;**. Siguiendo el ejemplo antes mencionado, la variable **x** recibe el resultado de la expresión **8 * (a + b)**.

2. Tokens del lenguaje.

Para que la implementación sea más simple el parser, primero se tokenizaran los elementos a categorías generales para que el parser solo cheque que los tokens están en el orden correcto de la gramática.

Elemento	Token
x, y, a, b, total, res	id
45, 3, 8	num
=	=
;	;
+	+
-	-
*	*
/	/
(	(
)	)

Siendo que una expresión como:

$x = 8 * (a + b);$

Se convierta en:

$id = num * (id + id);$

3. Gramática base con ambigüedad y recursividad izquierda

3.1 Ambigüedad:

Gramática base con posibilidad de generar las cadenas de esta gramática:

$S \rightarrow id = E;$

$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid id \mid num$

Esta gramática logra generar las cadenas del lenguaje, pero presenta ambigüedad, ya que una misma expresión puede interpretarse mediante más de un árbol.

En una gramática libre de contexto, la ambigüedad ocurre cuando una misma cadena puede tener más de una derivación o más de un árbol sintáctico válido. Esto sucede con la expresión $res = a + b * c;$, ya que puede interpretarse como $res = a + (b * c);$ o como $res = (a + b) * c;$. Sin embargo, en Javascript real la primera interpretación es la correcta, porque la multiplicación tiene mayor prioridad que la suma. (GeeksforGeeks, 2025)

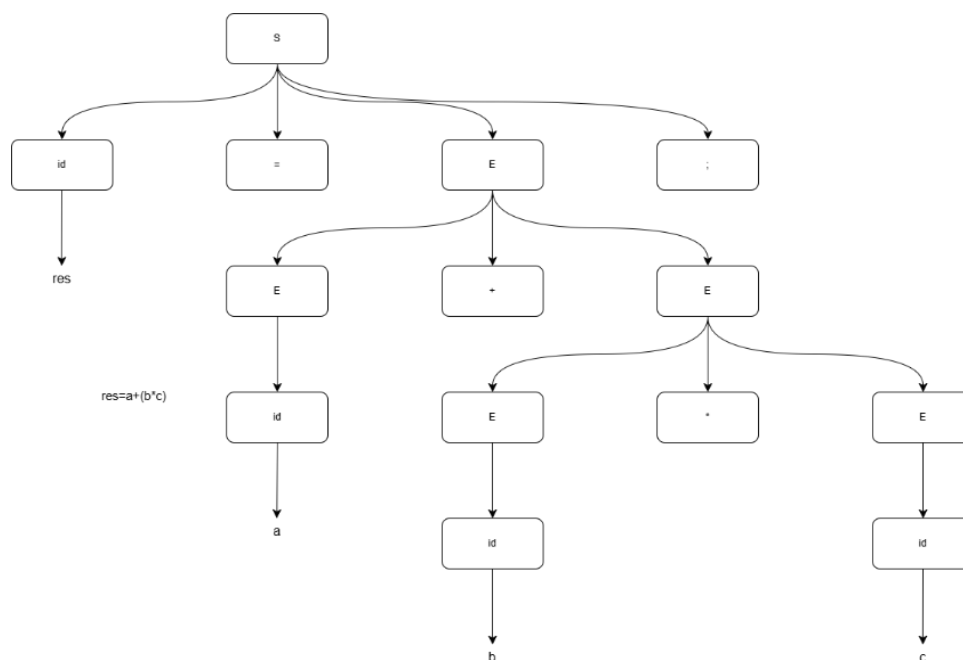


Figura 1. Diagrama de árbol correcto para la expresión $res = a + b * c;$.

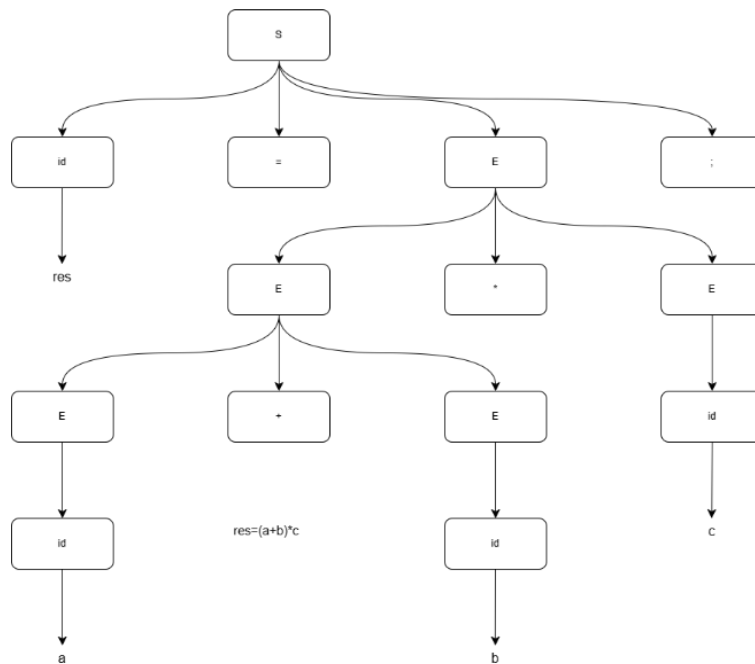


Figura 2. Diagrama de árbol incorrecto para la expresión $res = a + b * c; .$

Siendo la primera interpretación la correcta en Javascript real, ya que la multiplicación tiene mayor prioridad que la suma.

Para eliminar esta ambigüedad se añaden símbolos intermedios entre los actuales y sus valores, separando los elementos en su orden apropiado y dejando claras las reglas que debe seguir la gramática. En esta gramática de Javascript tiene mayor importancia los paréntesis, luego las multiplicación y divisiones que las sumas y restas. Por esto se dividirá en tres símbolos la gramática (sin contar terminales e inicial) quedando de la siguiente forma:

S -> id = E;
E -> E + T | E - T | T
T -> T * F | T / F | F
F -> (E) | **id** | **num**

Este orden ahora obliga a dar prioridad a multiplicaciones/divisiones dentro de paréntesis, sumas/restas dentro de paréntesis, multiplicaciones/divisiones fuera de paréntesis y por último sumas/restas fuera de paréntesis. Siendo que entre más bajo en el árbol primero es confirmado por el parser si esta correcto, al ser los últimos terminales a los que se llega.

Tomando como ejemplo la anterior expresión $res = a + b * c$, ahora el diagrama árbol con la nueva gramática quedaría de la siguiente forma:

Esta gramática ya fuerza la forma correcta de la expresión.

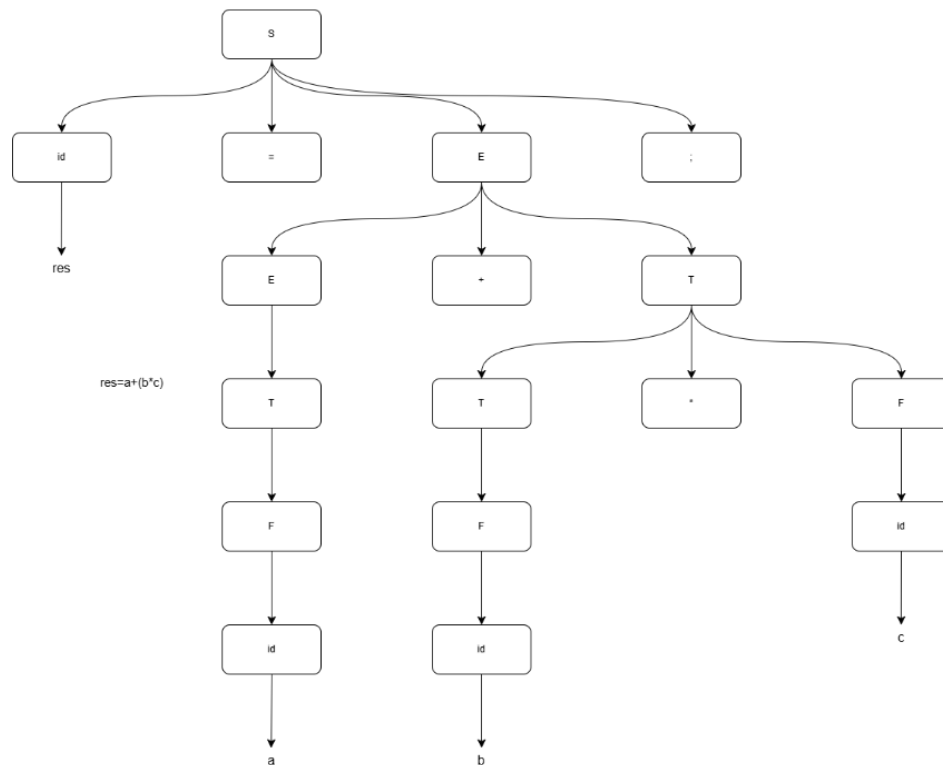


Figura 3. Diagrama de árbol correcto para la expresión $res = a + b * c$; sin ambigüedad.

3.2 Recursividad izquierda:

A pesar de haber eliminado la ambigüedad, la gramática todavía presenta recursividad izquierda en las reglas de **E** y **T**. Esto ocurre porque el mismo no terminal aparece al inicio de su propia producción, como en $E \rightarrow E + T$ o $T \rightarrow T * F$. Este tipo de recursividad puede causar problemas en parsers descendentes, ya que el analizador podría expandir el mismo símbolo indefinidamente sin consumir tokens de entrada. (GeeksforGeeks, 2025)

Para eliminar esta recursividad se usa la regla general en la que si se tiene:

$$A \rightarrow A\alpha \mid \beta$$

Se convierte en:

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

Por lo que se aplica esta regla tanto en **E** como en **T** que son los símbolos donde la recursividad izquierda está presente.

Antes:

$$E \rightarrow E + T \mid E - T \mid T$$

$T \rightarrow T * F \mid T / F \mid F$

Después:

$E \rightarrow T E'$

$E' \rightarrow - T E' \mid + T E' \mid \varepsilon$

$T \rightarrow F T'$

$T' \rightarrow / F T' \mid * F T' \mid \varepsilon$

Una vez acabadas estas modificaciones de de eliminación para recursión izquierda y ambigüedad queda la gramática final limpia:

S $\rightarrow$ id = E;

$E \rightarrow T E'$

$E' \rightarrow - T E' \mid + T E' \mid \varepsilon$

$T \rightarrow F T'$

$T' \rightarrow / F T' \mid * F T' \mid \varepsilon$

$F \rightarrow (E) \mid \text{id} \mid \text{num}$

Pudiendo generar expresiones como la propuesta al inicio $x = 8 * (a + b)$; sin ambigüedades ni recursividad izquierda:

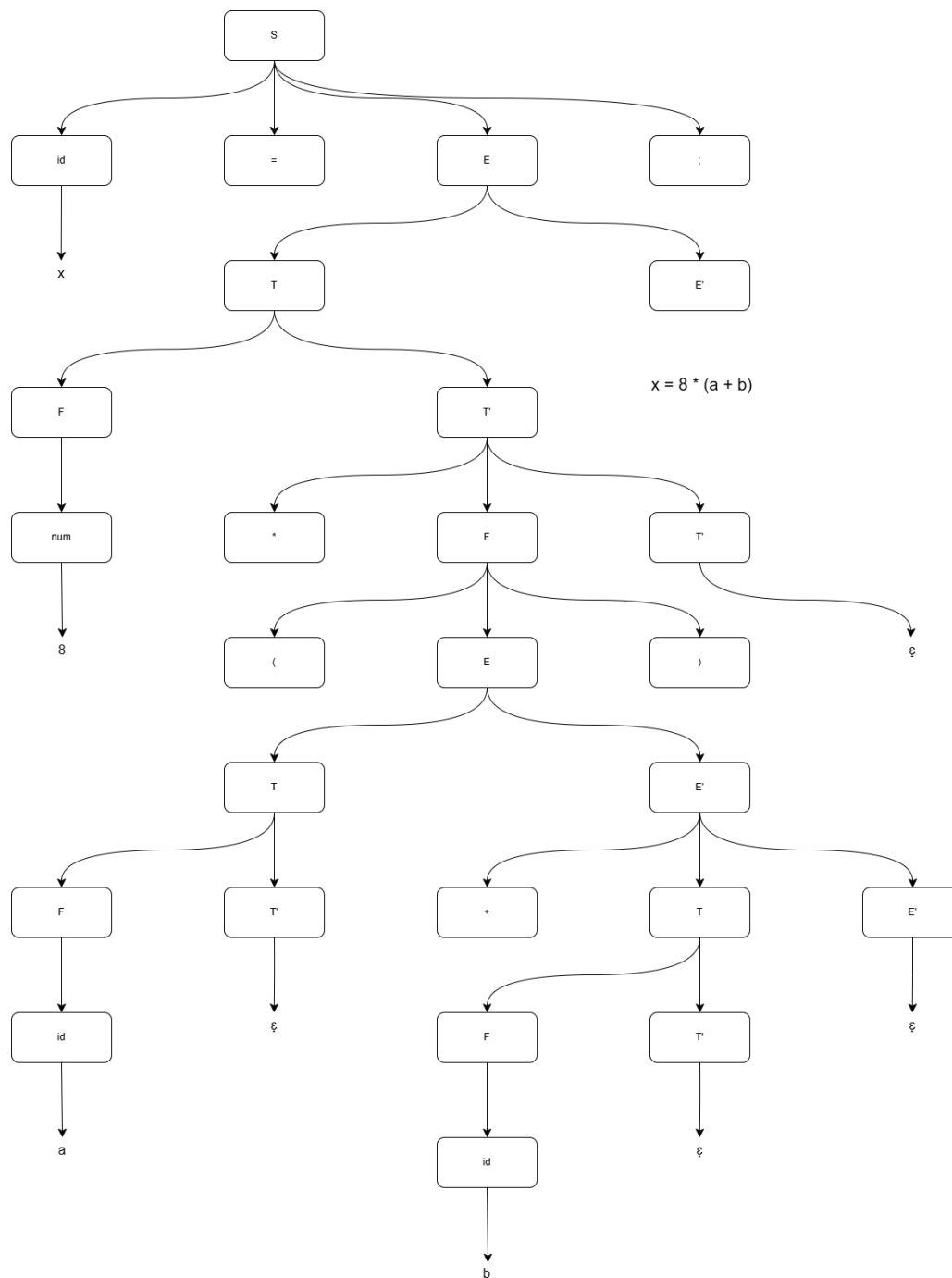


Figura 4. Diagrama de árbol correcto para la expresión $x = 8 * (a + b);$; sin ambigüedad y recursividad izquierda.

4. Documentación de pruebas

4.1 Casos aceptados:

1. $x = a + b;$

Tokens: id = id + id;

Diagrama árbol:

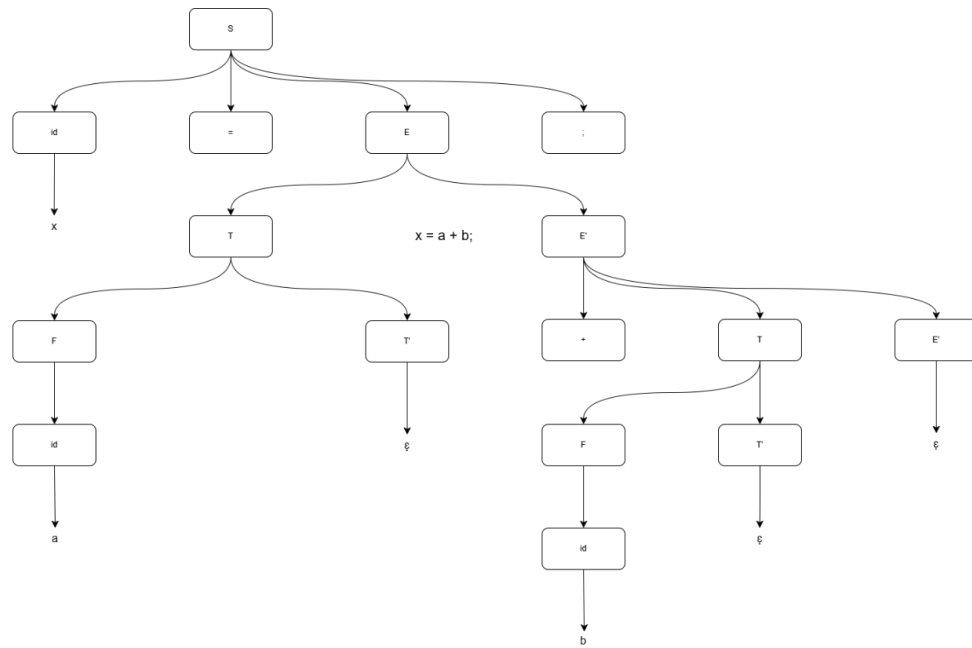


Figura 5. Diagrama de árbol para caso aceptado de la expresión $x = a + b;$.

Se acepta porque la cadena completa puede derivarse desde **S**.

2. $total = (c + 2) * y;$

Tokens: $id = (id + num) * id;$

Diagrama árbol:

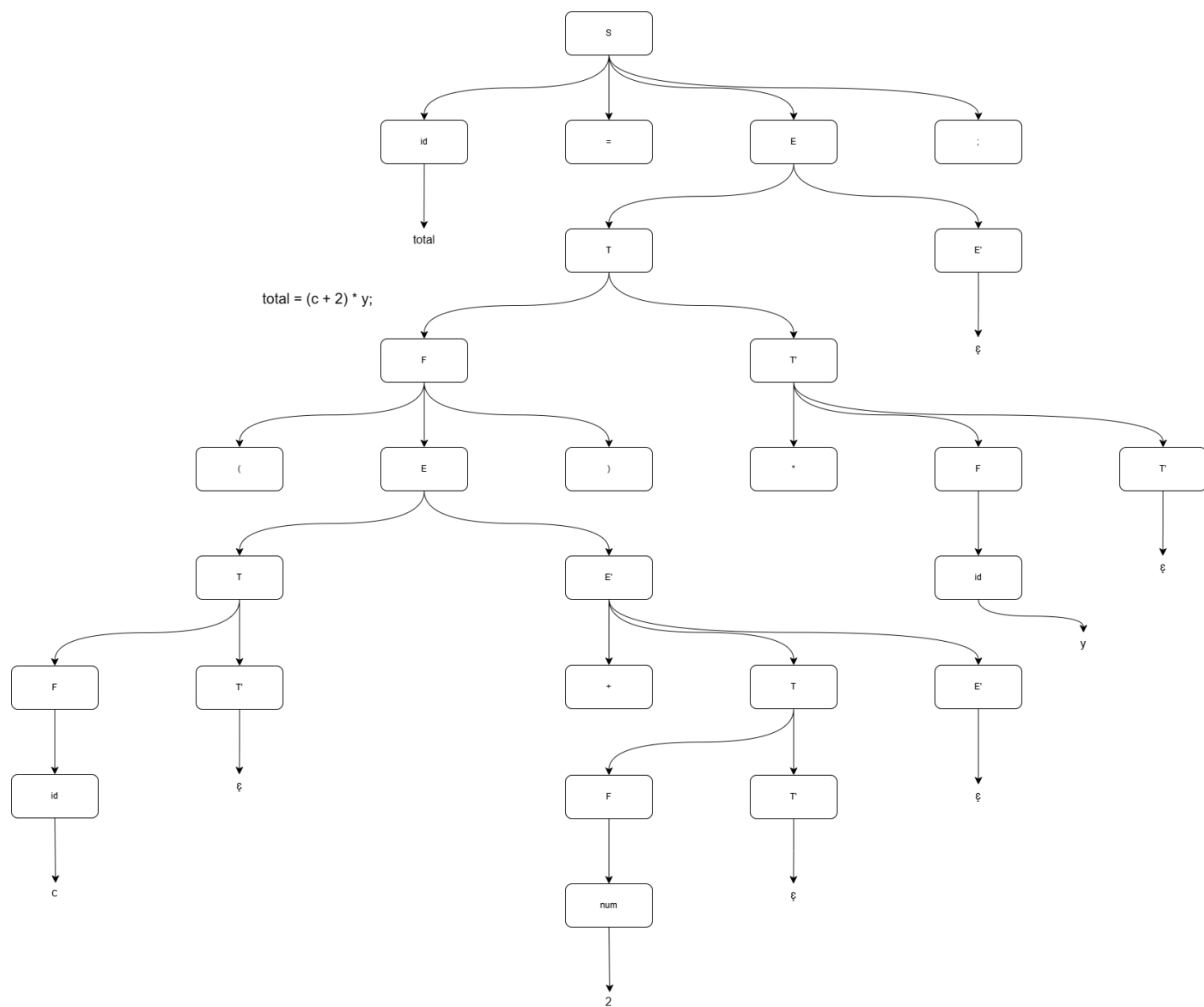


Figura 6. Diagrama de árbol para caso aceptado de la expresión $total = (c + 2) * y;$.

Se acepta porque la cadena completa puede derivarse desde **S**.

3. $resultado = num1 / num2;$

Tokens: id = id / id;

Diagrama árbol:

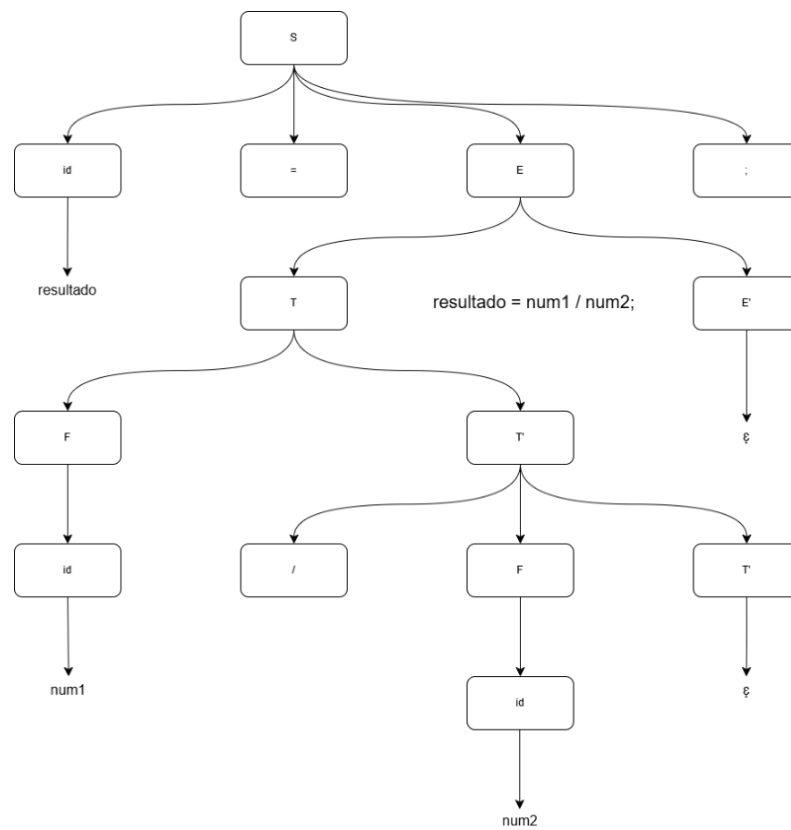


Figura 7. Diagrama de árbol para caso aceptado de la expresión `resultado = num1 / num2;`.

Se acepta porque la cadena completa puede derivarse desde **S**.

4. $x = (a);$

Tokens: `id = (id);`

Diagrama árbol:

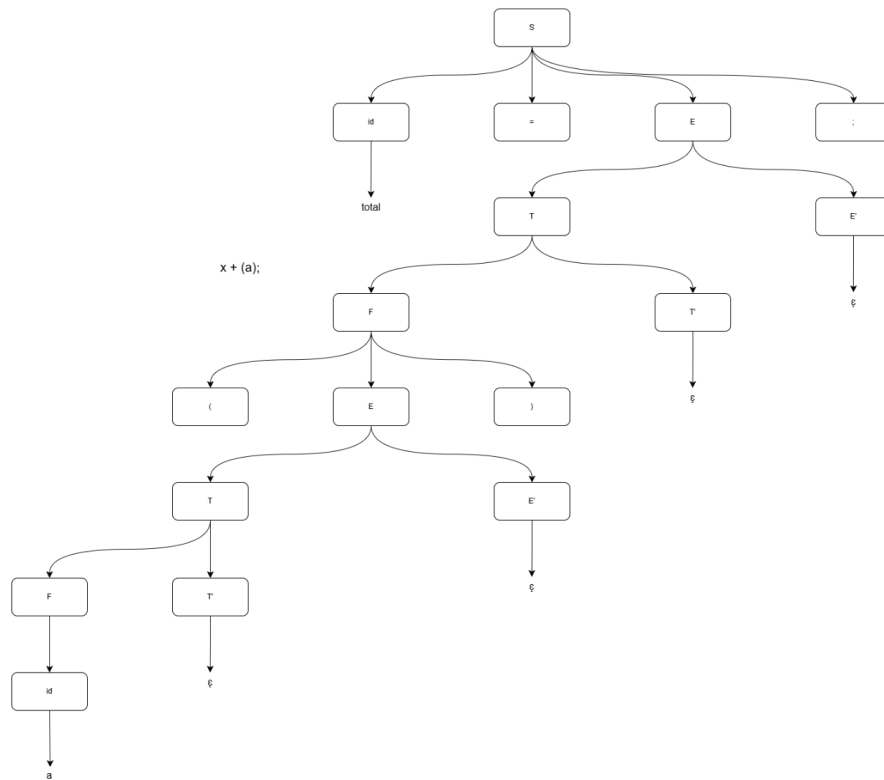


Figura 8. Diagrama de árbol para caso aceptado de la expresión $x = (a);$.

Se acepta porque la cadena completa puede derivarse desde **S**.

4.2 Casos rechazados:

1. $x = + a;$

Tokens: id = + id;

Después de leer **id =** la gramática esperaría un **id**, un **num** o un **(**, mientras que tiene un **+**. Por esto no hay un árbol válido y se rechaza la expresión.

2. $x a + b;$

Tokens: id id + id;

Después de leer **id** la gramática esperaría un **=**, mientras que tiene un **id**. Por esto no hay un árbol válido y se rechaza la expresión.

3. $x = a b;$

Tokens: id = id id;

Después de leer **id = id** la gramática esperaría un **+**, un **-**, un **/**, un *****, o un **;**, mientras que tiene un **id**. Por esto no hay un árbol válido y se rechaza la expresión.

4. $x = ();$

Tokens: id = ();

Después de leer **id = (** la gramática esperaría un **id**, un **num** o un **(**, mientras que tiene un **)**. Por esto no hay un árbol válido y se rechaza la expresión.

5. Nivel en jerarquía de Chomsky

5.1 Antes de limpiar:

La gramática base es la siguiente:

$S \rightarrow id = E;$

$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid id \mid num$

Esta gramática pertenece al Tipo 2 de la jerarquía de Chomsky, es decir, es una gramática libre de contexto. Aunque sea ambigua y tenga recursividad izquierda, sigue siendo libre de contexto porque sus producciones mantienen un solo símbolo no terminal del lado izquierdo. En la jerarquía de Chomsky, las gramáticas libres de contexto se caracterizan precisamente por reglas donde un no terminal puede producir una cadena de terminales y no terminales. Por ello, la gramática base de expresiones aritméticas sigue siendo Tipo 2 antes de limpiarla. (GeeksforGeeks, 2026)

Aparte como se vio durante clase un ejemplo de libre de contexto son las expresiones aritméticas que son las que se modelan aquí pero con su versión correcta para Javascript.

5.2 Después de limpiar:

La gramática final después de quitar ambigüedad y recursividad izquierda es la siguiente:

$S \rightarrow id = E;$

$E \rightarrow T E'$

$E' \rightarrow - T E' \mid + T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow / F T' \mid * F T' \mid \epsilon$

$F \rightarrow (E) \mid id \mid num$

Después de eliminar la ambigüedad y la recursividad izquierda, la gramática sigue perteneciendo al Tipo 2 de la jerarquía de Chomsky, ya que sus reglas continúan teniendo un solo no terminal en el lado izquierdo. La limpieza de la gramática no cambia su clasificación formal, lo que cambia es su utilidad para el análisis sintáctico, porque ahora la gramática es más clara y evita interpretaciones múltiples. (GeeksforGeeks, 2026)

6. Complejidad temporal

6.1 Antes de limpiar:

Al la gramática base ser ambigua, el parser tendría que considerar distintos posibles árboles para una misma cadena, como fue visto en la sección **3.1 de ambigüedad**. Este problema solo crecerá entre más larga sea la expresión a analizar, por lo que el análisis de una gramática ambigua sería muy costoso. Aparte de que al no solo ser ambigua pero también con recursividad izquierda podría expandir infinitamente **E** sin llegar a algún resultado.

6.2 Después de limpiar:

Una vez que la gramática está libre de ambigüedad y recursividad izquierda, el análisis puede realizarse de manera más directa, ya que el parser no necesita considerar múltiples árboles para una misma cadena ni corre el riesgo de expandir indefinidamente una producción izquierda recursiva. Para una cadena de longitud n , un parser predictivo como **LL(1)** puede analizar la entrada en tiempo lineal $O(n)$, porque lee los tokens de izquierda a derecha y decide qué producción aplicar con base en el símbolo actual. En este proyecto, la implementación con NLTK permite comprobar esta estructura generando árboles válidos únicamente para las cadenas que pertenecen a la gramática. (GeeksforGeeks, 2025; NLTK Project, n.d.)

7. Referencias:

- GeeksforGeeks. (2025). Ambiguity in context free grammar and context free languages.
<https://www.geeksforgeeks.org/ambiguity-in-context-free-grammar-and-context-free-languages/>
- GeeksforGeeks. (2025). Removing direct and indirect left recursion in a grammar.
<https://www.geeksforgeeks.org/dsa/removing-direct-and-indirect-left-recursion-in-a-grammar/>
- GeeksforGeeks. (2026). Chomsky hierarchy in theory of computation.
<https://www.geeksforgeeks.org/theory-of-computation/chomsky-hierarchy-in-theory-of-computation/>
- NLTK Project. (n.d.). nltk.grammar.CFG.
<https://www.nltk.org/api/nltk.grammar.CFG.html>
- NLTK Project. (n.d.). nltk.parse.chart module.
<https://www.nltk.org/api/nltk.parse.chart.html>